CareQueue Technical Coding Study Guide

Deep technical notes for frontend developer fresher interviews

Purpose

This PDF explains the coding side of CareQueue: React root setup, routing, protected routes, Context API, localStorage, forms, filtering logic, booking data structure, deployment root directory, and technical interview answers.

Topic	What you should study
React root	How index.html, main.jsx, and createRoot connect the React app to the browser.
Routes	How BrowserRouter, Routes, Route, useNavigate, and useSearchParams are used.
Auth	How AuthContext, AuthProvider, useAuth, login, logout, and protected rendering work.
localStorage	How users, currentUser, and carequeue_bookings are stored and read.
Forms	How controlled inputs collect appointment data.
Data logic	How arrays, objects, filter, find, map, optional chaining, and helper functions are used.
Deployment	How Vite build, dist output, Vercel root directory, and vercel.json rewrites work.

1. Project File Structure

CareQueue is organized as a Vite React project. The main code lives inside src, reusable UI screens are inside components, and mock business data lives inside data.

```
carequeue/
index.html
package.json
vite.config.js
vercel.json
src/
main.jsx
App.jsx
index.css
components/
AuthPage.jsx
AuthContext.jsx
AuthContextValue.js
useAuth.js
LandingPage.jsx
BookAppointments.jsx
SearchResults.jsx
BookingForm.jsx
MyAppointments.jsx
Navbar.jsx
Footer.jsx
data/
pricing.js
consultations.js
```

File	Technical role
index.html	Contains the div with id root. React injects the app into this element.
src/main.jsx	Creates the React root and renders App.
src/App.jsx	Defines routes and protected page rendering.
AuthContext.jsx	Stores login state and login/logout functions.
AuthPage.jsx	Handles signup and login form logic.
SearchResults.jsx	Reads query params and filters data.
BookingForm.jsx	Handles controlled form state and stores booking records.
MyAppointments.jsx	Reads bookings and updates cancelled status.
pricing.js	Maps services and centres to prices.
consultations.js	Stores consultation, hospital, doctor, and review mock data.

2. React Root Setup

There are three root concepts you should explain clearly: HTML root, React root, and route root.

HTML root in index.html

```
<div id="root"></div>
```

React searches for this element and renders the complete app inside it.

React root in main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
```

```
<StrictMode>
<App />
</StrictMode>,
)
```

- createRoot is the React 18+ API for creating a root rendering container.
- document.getElementById('root') selects the div from index.html.
- StrictMode helps detect possible React issues during development.
- App is the top-level component where routing and providers are added.

Route root in App.jsx

```
<Route path="/" element={isLoggedIn ? <LandingPage /> : <AuthPage />} />
```

The / route is the home route of the SPA. In this project it shows LandingPage only after login.

3. Routing and Protected Routes

CareQueue uses BrowserRouter from react-router-dom. BrowserRouter enables clean URLs like /search and /appointments.

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

const AppContent = () => {
const { isLoggedIn } = useAuth();

return (
<Routes>
<Route path="/" element={isLoggedIn ? <LandingPage /> : <AuthPage />} />
<Route path="/search" element={isLoggedIn ? <SearchResults /> : <AuthPage />} />
<Route path="/booking-form" element={isLoggedIn ? <BookingForm /> : <AuthPage />} />
<Route path="/appointments" element={isLoggedIn ? <MyAppointments /> : <AuthPage />} />
</Routes>
);
};
```

Technique used: protected rendering

- The app checks isLoggedIn before rendering each page.
- If true, the requested page is shown.
- If false, AuthPage is shown instead.
- This is a simple frontend route protection pattern.
- For production, backend/session validation is needed because frontend-only protection can be bypassed.

Navigation using useNavigate

```
const navigate = useNavigate();

const handleBook = (service) => {
navigate(`/search?service=${encodeURIComponent(service)}`);
};
```

useNavigate is used to move users programmatically after button clicks, login, booking, or service selection.

4. URL Query Params

CareQueue passes selected service, centre, and doctor through URL query parameters. This keeps the page shareable and avoids global state for selected service details.

Creating query params

```
navigate(
`/booking-form?center=${encodeURIComponent(center.name)}&service=${encodeURIComponent(service)}`
);
```

Reading query params

```
const [searchParams] = useSearchParams();

const centerName = searchParams.get('center') || 'Selected Center';
const serviceName = searchParams.get('service') || 'General Checkup';
const doctorName = searchParams.get('doctor') || null;
```

Technical point

encodeURIComponent is used because service names can contain spaces, such as blood tests or cardiologist consultation. It safely converts text into a URL-friendly format.

5. Context API and Custom Hook

The app uses Context API to avoid passing login state manually through many components.

Creating context

```
import { createContext } from "react";

export const AuthContext = createContext();
```

Providing context values

```
export const AuthProvider = ({ children }) => {
const [isLoggedIn, setIsLoggedIn] = useState(false);

const login = (user) => {
localStorage.setItem("currentUser", JSON.stringify(user));
setIsLoggedIn(true);
};

const logout = () => {
localStorage.removeItem("currentUser");
setIsLoggedIn(false);
};

return (
<AuthContext.Provider value={{ isLoggedIn, login, logout }}>
{children}
</AuthContext.Provider>
);
};
```

Custom hook

```
import { useContext } from "react";
import { AuthContext } from "./AuthContextValue";

export const useAuth = () => useContext(AuthContext);
```

- AuthContext stores shared authentication data.
- AuthProvider wraps the whole app in App.jsx.
- useAuth is a custom hook that makes context access cleaner.
- Components can call const { isLoggedIn, login, logout } = useAuth().

6. localStorage Deep Dive

localStorage is browser storage. It stores string key-value pairs and persists even after page refresh. CareQueue uses it to simulate backend persistence.

Key	Value stored
users	Array of signup users, saved as JSON string.
currentUser	Logged-in user object, saved as JSON string.
carequeue_bookings	Array of appointment objects.

Important localStorage methods

```
localStorage.setItem("key", "value"); // save
localStorage.getItem("key"); // read
localStorage.removeItem("key"); // delete

JSON.stringify(objectOrArray); // convert JS data to string
JSON.parse(stringValue); // convert string back to JS data
```

Signup storage pattern

```
let users = JSON.parse(localStorage.getItem("users")) || [];

const exists = users.find((u) => u.email === user.email);
if (exists) {
alert("User already exists!");
return;
}

users.push(user);
localStorage.setItem("users", JSON.stringify(users));
```

Login pattern

```
let users = JSON.parse(localStorage.getItem("users")) || [];

const validUser = users.find(
(u) => u.email === user.email && u.password === user.password
);

if (validUser) {
login(validUser);
navigate("/");
}
```

Booking storage pattern

```
const currentUser = JSON.parse(localStorage.getItem("currentUser"));
const bookedAt = new Date().toISOString();

const booking = {
...formData,
id: crypto.randomUUID?.() || bookedAt,
centerName,
price: priceDetails,
userEmail: currentUser?.email || "unknown",
status: "Confirmed",
bookedAt,
};

const existing = JSON.parse(localStorage.getItem("carequeue_bookings") || "[]");
existing.push(booking);
localStorage.setItem("carequeue_bookings", JSON.stringify(existing));
```

Interview caution

localStorage is not secure for passwords or medical data. In interviews, say you used it only to simulate persistence for a frontend prototype.

7. React State and Controlled Forms

CareQueue forms are controlled forms. The input values are stored in React state and updated through onChange.

```
const [formData, setFormData] = useState({
name: '',
age: '',
mobile: '',
testPrefer: serviceName,
appointmentDate: '',
timing: '',
bookingFor: 'myself',
patientName: '',
patientContact: ''
});
```

Generic handleChange

```
const handleChange = (e) => {
const { name, value } = e.target;
setFormData(prev => ({
...prev,
[name]: value
}));
};
```

- name comes from the input name attribute.
- value comes from the user's typed/selected value.
- ...prev keeps all existing form fields unchanged.
- [name]: value updates only the changed field dynamically.
- This avoids writing separate state functions for every input.

Input connected to state

```
<input
type="text"
name="name"
value={formData.name}
onChange={handleChange}
required
/>
```

8. Array and Object Techniques

The project uses common JavaScript data techniques that are important for frontend interviews.

Technique	Where used
find	Find matching user during login and duplicate email during signup.
filter	Filter diagnostic centres by selected service and filter bookings by current user email.
map	Render cards for centres, doctors, reviews, services, and appointments.
some	Check whether a centre provides the selected service.
optional chaining	Safely access currentUser?.email or HOSPITAL_INFO[centerName]?.rating.
spread operator	Copy formData into booking object and copy previous state during form updates.
computed property	[name]: value updates the correct form field dynamically.

Filtering centres

```
filteredCenters = MOCK_CENTERS.filter(center =>
center.services.some(
s => s.toLowerCase() === service.toLowerCase()
)
);
```

Rendering list with map

```
{filteredCenters.map(center => (
<div key={center.id}>
<h2>{center.name}</h2>
<p>{center.location}</p>
</div>
))}
```

Filtering user bookings

```
const getUserBookings = (email) => {
const allBookings = JSON.parse(
localStorage.getItem("carequeue_bookings") || "[]"
);
return allBookings.filter((booking) => booking.userEmail === email);
};
```

9. Conditional Rendering

Conditional rendering means showing different UI based on state or data.

Login protection

```
element={isLoggedIn ? <LandingPage /> : <AuthPage />}
```

Consultation vs diagnostic results

```
const isConsultation =
CONSULTATION_TYPES.includes(service.toLowerCase());

return (
<>
{isConsultation ? (
<ConsultationResults />
) : (
<DiagnosticCenterResults />
)}
</>
);
```

Booking for myself vs someone else

```
{formData.bookingFor === 'someone_else' && (
<div>
<input name="patientName" />
<input name="patientContact" />
</div>
)}
```

Submitted confirmation

```
if (submitted) {
return <BookingConfirmation />;
}
```

10. Pricing Helper Logic

The pricing file separates business rules from UI components. This makes the code easier to maintain.

```
const getPriceServiceType = (serviceName) => {
const normalized = serviceName.toLowerCase();
if (normalized.includes("nuchal")) return "nuchal";
if (normalized.includes("echo")) return "echocardiogram";
if (normalized.includes("ecg")) return "ecg";
if (normalized.includes("ultrasound")) return "ultrasound";
if (normalized.includes("x-ray")) return "x-ray";
if (normalized.includes("blood")) return "blood";
if (normalized.includes("scan")) return "advanced-scan";
return "general";
};
```

```
export const getPriceDetails = (centerName, serviceName) => {
const type = getPriceServiceType(serviceName);
return CENTER_PRICES[centerName]?.[type] || DEFAULT_PRICES[type];
};
```

- The service name is normalized to lowercase.
- includes checks whether the service belongs to a known category.
- The centre-specific price is checked first.
- If centre-specific price is missing, DEFAULT_PRICES is used.
- Optional chaining prevents errors when a centre does not exist in CENTER_PRICES.

11. Appointment Cancellation Logic

Cancelling does not delete the booking. It updates status from Confirmed to Cancelled. This is better for history tracking.

```
const handleCancel = (bookingId) => {
const allBookings = JSON.parse(
localStorage.getItem("carequeue_bookings") || "[]"
);

const updated = allBookings.map((b) =>
b.id === bookingId ? { ...b, status: "Cancelled" } : b
);

localStorage.setItem("carequeue_bookings", JSON.stringify(updated));
setAppointments(getUserBookings(currentUser?.email));
};
```

- map creates a new updated bookings array.
- The matching booking is copied with spread operator and status changed.
- Other bookings are returned unchanged.
- The updated array is saved back into localStorage.
- React state is updated so the UI refreshes immediately.

12. Vite, Build, and Deployment

The app is built using Vite. Vite converts React source files into optimized static files inside dist.

package.json scripts

```
"scripts": {
"dev": "vite",
"build": "vite build",
"lint": "eslint .",
"preview": "vite preview"
}
```

Command	Meaning
npm run dev	Starts local development server.
npm run build	Creates production build in dist.
npm run lint	Checks code quality using ESLint.
npm run preview	Runs a local preview of the production build.

Vercel root directory

Your GitHub repository contains the app inside a nested folder. That is why the Vercel Root Directory must point to the folder containing package.json.

```
React/CareQueue_project/carequeue
```

How to explain root directory

Vercel runs npm install and npm run build from the selected root directory. If the root directory is wrong, Vercel cannot find package.json and deployment fails.

vercel.json SPA rewrite

```
{
"rewrites": [
{
"source": "/(.*)",
"destination": "/index.html"
}
]
}
```

This makes direct refreshes on /search, /appointments, and /about work with BrowserRouter.

13. Technical Interview Answers

Question	Answer
What is the root of your React app?	index.html has a div with id root. main.jsx uses createRoot(document.getElementById('root')) to render App inside it.
How did you protect routes?	I used isLoggedIn from AuthContext and conditionally rendered either the actual page or AuthPage for each route.
How does localStorage work?	localStorage stores string values in the browser. I use JSON.stringify to save arrays/objects and JSON.parse to read them back.
Why did you use Context API?	To share authentication state and login/logout functions globally without prop drilling.
What is a controlled component?	A form input whose value is controlled by React state and updated with onChange.
How do you pass selected service?	I pass it through URL query params using useNavigate and read it using useSearchParams.
How does filtering work?	I use filter and some to match selected services with centre service arrays.
How are appointments displayed?	I read all bookings from localStorage and filter by currentUser email.
Why use Vercel rewrite?	React Router routes are client-side routes, so Vercel must serve index.html for all paths.
What is the biggest limitation?	No backend or database. localStorage is only for demo persistence and is not secure for production.

14. Code Concepts Checklist

- React components and JSX
- Vite project setup and build pipeline
- ReactDOM createRoot rendering
- BrowserRouter, Routes, Route
- useNavigate for programmatic navigation
- useSearchParams for URL query params
- useState for form and UI state

- useEffect for checking saved login state
- Context API for global auth state
- Custom hook useAuth
- localStorage setItem, getItem, removeItem
- JSON.stringify and JSON.parse
- Array methods: find, filter, map, some
- Object spread and computed property names
- Conditional rendering with ternary and &&
- Controlled forms with value and onChange
- Vercel root directory and SPA rewrite config

Final technical summary

CareQueue is a Vite React SPA. The app root is mounted through main.jsx, pages are handled by React Router, login state is shared through Context API, form data is managed with useState, mock persistence is handled through localStorage, service results are generated using array filtering, and deployment is handled through Vercel using the correct root directory and SPA rewrite.